MODULE 17 · DOMAIN 2 - SECURITY

Amazon Cognito

User pools sign people in. Identity pools hand out AWS credentials. Almost every question turns on that distinction.

DOMAIN 2 of 4 EXAM WEIGHT 26% MODULE 17 of 34

admin@sthithapragna.com

A user pool answers **who is this person**. An identity pool answers **what may they do in AWS**.

User pool or identity pool?

USER POOL

Authentication

- A directory of users you own
- Sign-up, sign-in, password reset, MFA
- Returns JWTs, not AWS credentials
- Authorises API Gateway and ALB directly
- Federates SAML and social providers in

IDENTITY POOL

AWS credentials

- Holds no users of its own
- Exchanges a token for STS credentials
- Returns access key, secret and session token
- Lets a client call S3 or DynamoDB directly
- Supports unauthenticated guest access

THE TIEBREAKER

If the app calls an API you built, a user pool alone is enough. If it calls an AWS service directly from the client, you need an identity pool too.

User pools

- Sign-up, sign-in, email and phone verification, password policy and account recovery.
- Standard attributes such as email and name, plus **custom attributes** prefixed custom:.
- Custom attributes are fixed at creation. You cannot rename or remove one afterwards.
- **Groups** carry an IAM role and a **precedence** number, and appear in the token.
- External providers can federate in, so one pool serves local and corporate users alike.

> ON THE EXAM
>
> The custom attribute constraint is a real design decision. Adding one is easy; changing your mind means a new pool and a migration.

The three tokens

TOKEN	WHAT IT IS FOR
ID token	Proves identity. Carries user attributes as claims. For your application
Access token	Proves authorisation. Carries scopes and groups. For APIs and resource servers
Refresh token	Obtains new ID and access tokens. Opaque, not a JWT, cannot be decoded

ON THE EXAM

ID and access tokens are both JWTs you can decode and inspect. The refresh token is opaque, which is why you cannot read its expiry from the token itself.

Token lifetimes

- **ID and access tokens:** 5 minutes to 1 day. Default **1 hour**.
- **Refresh token:** 60 minutes to **10 years**. Default **30 days**.
- An access token cannot outlive its refresh token. The console enforces that.
- Configured **per app client**, so a mobile app and a web app can differ.
- **RevokeToken** invalidates a refresh token and every access token issued from it.

> ON THE EXAM
>
> Long refresh tokens mean a signed-out or deactivated user can keep working. RevokeToken is how you stop that without waiting for expiry.

ID token or access token?

ID TOKEN

Who the user is

- Contains email, name, custom attributes
- Audience is your application
- Used to personalise the interface
- Should not be sent to third-party APIs
- Analogous to an identity card

ACCESS TOKEN

What they may do

- Contains scopes and cognito:groups
- Audience is a resource server
- Used to authorise API calls
- The correct token for an API to validate
- Analogous to a ticket

THE TIEBREAKER

Sending an ID token to an API is the classic mistake: it leaks user attributes and was never intended as an authorisation credential.

Validating a token

- Verify the **signature** against the pool's public keys, published at the **jwks_uri** endpoint.
- Check **exp** for expiry, **iss** for the issuing pool, and **aud** or client_id for the app client.
- Check **token_use**: it should be **access** for an API, and **id** for your application.
- Cache the JWKS. Fetching it on every request is a needless round trip.
- API Gateway's Cognito authoriser and ALB authentication do all of this for you.

> ON THE EXAM
>
> Decoding a JWT without verifying the signature is not validation. Anyone can craft a token whose claims say anything at all.

App clients and OAuth flows

- An **app client** represents one application. It sets callback URLs, scopes and token lifetimes.
- Public clients, such as mobile and browser apps, must have **no client secret**.
- **Authorization code grant** with PKCE is the correct flow for user-facing apps.
- **Implicit grant** returns tokens in the URL fragment and is deprecated. Do not choose it.
- **Client credentials** is for machine-to-machine. It returns an access token and no user.

> ON THE EXAM
>
> Machine-to-machine means client credentials, and there is no ID token because there is no user. That absence is itself an exam signal.

Server-side and admin APIs

- **InitiateAuth** and friends are called by the client, with no AWS credentials needed.
- **AdminInitiateAuth** and the other Admin operations are called by your backend, with IAM credentials.
- Admin operations can create, disable, delete and reset users without their involvement.
- **ADMIN_USER_PASSWORD_AUTH** sends the password to your server first. Only use it when the client cannot use SRP.
- **SRP** is the default flow, and never transmits the password at all.

> ON THE EXAM
>
> The Admin prefix is the tell: those calls require AWS credentials, so they can only come from a backend, never from a browser or a device.

Currency note: Managed Login and tiers

- **The examinable term:** the **hosted UI**, a Cognito-provided sign-in page.
- **Managed Login** replaced it, adding a no-code branding editor.
- **Passwordless sign-in** arrived at the same time: passkeys, email OTP and SMS OTP.
- User pools now have tiers: **Lite**, **Essentials** and **Plus**. Essentials is the default.
- **Advanced security features** were renamed **threat protection** and moved into the Plus tier.

> READ THIS TWICE
>
> The rename matters most. Exam material says 'advanced security features'; the console says 'threat protection'. They are the same capability.

Federating into a user pool

- A user pool can trust **social providers**, **SAML 2.0** providers and generic **OIDC** providers.
- Federated users appear in the pool as normal users, with an **identities** attribute.
- Your application receives Cognito tokens either way, so it never handles multiple formats.
- **Attribute mapping** copies claims from the provider into pool attributes.
- Provider profile changes do not propagate automatically. They map at sign-in.

> ON THE EXAM
>
> This is the real value of a user pool as a front door: your code deals with one token format regardless of how the user authenticated.

User pool domains

- Managed Login needs a domain: either a Cognito prefix domain or one of your own.
- A Cognito domain looks like **prefix.auth.region.amazoncognito.com** and needs no setup.
- A **custom domain** such as auth.example.com looks like your brand and needs an ACM certificate.
- That certificate must be in **us-east-1**, because a CloudFront distribution fronts it.
- Callback and sign-out URLs are configured per app client, and must match exactly.

> ON THE EXAM
>
> The us-east-1 certificate requirement is the same gotcha as an edge-optimised API Gateway custom domain, and for the same reason: CloudFront.

MFA and threat protection

- MFA can be **off**, **optional** per user, or **required** for everyone.
- Factors: SMS, **TOTP** authenticator apps, and email. Passkeys cover the passwordless case.
- **Adaptive authentication** scores each sign-in for risk and can step up to MFA.
- **Compromised credentials detection** checks credentials against known breach data.
- Both of those sit in the **Plus** tier, formerly advanced security features.

> ON THE EXAM
>
> The exam framing: risk-based step-up authentication and breach checking are a paid tier, not a default. Basic MFA is available everywhere.

Lambda triggers

TRIGGER	USE IT TO
Pre sign-up	Auto-confirm users, or reject sign-ups from unapproved domains
Post confirmation	Create the user's record in your own database
Pre authentication	Block a sign-in based on your own rules
Post authentication	Log the sign-in, or update a last-seen timestamp
Pre token generation	Add or modify claims in the token
Migrate user	Authenticate against a legacy system and import the user
Custom message	Rewrite verification and invitation emails and SMS
Define, Create and Verify auth challenge	Build a custom authentication flow

ON THE EXAM

These are the most examinable part of user pools. Each answers a specific 'how do I hook into sign-in' question, and the names are self-describing.

The pre token generation trigger

- Runs just before tokens are issued, and can **add, modify or suppress claims**.
- The standard way to inject a **tenant ID**, a role, or entitlements into a token.
- Those claims then reach your backend, so it needs no extra database lookup per request.
- It can also modify group membership for that session.
- Adding claims grows the token. Very large tokens can exceed header size limits.

> ON THE EXAM
>
> This is how multi-tenant applications avoid a lookup on every request: the tenant is already in the verified token, and the signature makes it trustworthy.

The user migration trigger

- The problem: you have an existing user database, but you only store password **hashes**.
- Hashes cannot be imported, so a bulk import would force everyone to reset their password.
- The trigger fires when an unknown user attempts to sign in.
- Your function authenticates them against the old system and returns their attributes.
- Cognito then creates the user, using the password they just typed. Migration is invisible.

> ON THE EXAM
>
> This is the answer whenever a scenario says users must not be forced to reset their passwords during a migration to Cognito.

Custom authentication flows

- **Define auth challenge** decides what challenge comes next, or that authentication is complete.
- **Create auth challenge** generates the challenge, such as sending a one-time code.
- **Verify auth challenge response** checks the answer the user gave.
- Together these build passwordless flows, CAPTCHA steps, or bespoke multi-step sign-in.
- Since November 2024, native passwordless options exist, so custom flows are needed less often.

ON THE EXAM

The three run in a loop: define decides, create issues, verify checks, then define decides again. That loop is the whole mechanism.

Groups and precedence

- A group can carry an **IAM role** and a numeric **precedence**.
- Membership appears in the token as the **cognito:groups** claim.
- Your API can authorise on that claim without any additional lookup.
- **Lower precedence wins.** Zero outranks one, which is the opposite of most intuitions.
- With an identity pool, the role of the highest-priority group is the one selected.

> ON THE EXAM
>
> Groups give you role-based access control for free. Anything finer than that needs claims, an authoriser, or Verified Permissions.

Identity pools

- Takes a token from a trusted provider and returns **temporary AWS credentials**.
- Providers: a Cognito user pool, social providers, SAML, OIDC, or a developer-authenticated identity.
- Two roles are configured: one for **authenticated** users, one for **unauthenticated** guests.
- Guest access is genuinely useful: browse before signing in, with a deliberately narrow role.
- The credentials are ordinary STS credentials, so any AWS SDK uses them normally.

> ON THE EXAM
>
> An identity pool is a managed wrapper around AssumeRoleWithWebIdentity from module 16. It is the same exchange, with the plumbing done for you.

Choosing the role

- **Default roles** give every authenticated user the same role. Simple, and often too broad.
- **Rules-based** selection matches token claims and picks a role accordingly.
- **Token-based** selection uses the **cognito:preferred_role** claim from group membership.
- Role policies can use **policy variables** so each user reaches only their own data.
- **${cognito-identity.amazonaws.com:sub}** is the identity ID, and scopes an S3 prefix per user.

> ON THE EXAM
>
> That last variable is the standard per-user isolation pattern: one bucket, one policy, and every user confined to their own prefix.

Basic and enhanced flows

- **Enhanced flow** is the default: one call to **GetCredentialsForIdentity**.
- It always issues **1-hour** credentials. There is no duration parameter.
- **Basic flow** is three calls: GetId, GetOpenIdToken, then AssumeRoleWithWebIdentity.
- Because the last call is yours, you can request up to the role's maximum session duration.
- Basic flow is disabled by default and must be enabled on the identity pool.

> ON THE EXAM
>
> If a scenario needs identity pool credentials to last longer than an hour, the enhanced flow cannot do it. Basic flow plus a longer role session duration is the answer.

The full flow, end to end

01

Sign in

The user authenticates against the user pool and receives three tokens.

02

Exchange

The app passes the ID token to the identity pool.

03

Credentials

The identity pool returns temporary AWS credentials for a role.

04

Call AWS

The app calls S3 or DynamoDB directly, scoped by the role's policy.

ON THE EXAM

Note step 2 uses the ID token, not the access token. The identity pool is establishing who the user is, which is exactly what the ID token proves.

Cognito with API Gateway

- A **Cognito user pool authoriser** on a REST API validates the token for you. No Lambda.
- Claims reach the backend through **$context.authorizer.claims**.
- HTTP APIs use a **JWT authoriser**, which works with Cognito or any OIDC provider.
- Neither does per-resource authorisation. They answer only 'is this a valid user'.
- For finer rules, use a **Lambda authoriser** or **Amazon Verified Permissions**.

> ON THE EXAM
>
> Module 11 made the same point from the API Gateway side. The Cognito authoriser is authentication; anything conditional on the resource needs more.

Cognito beyond API Gateway

- An **Application Load Balancer** can authenticate users against a user pool before forwarding.
- That gives sign-in to an existing application with no code change at all.
- **AWS AppSync** accepts Cognito user pools as an authorisation mode for GraphQL.
- **Amazon S3** and other services are reached through identity pool credentials, not tokens.
- The ALB passes user claims to the backend in **x-amzn-oidc** headers.

> ON THE EXAM
>
> ALB authentication is the answer when a scenario wants sign-in in front of a legacy application that cannot be modified.

Amazon Verified Permissions

- A managed authorisation service using the **Cedar** policy language.
- Answers questions like: may **this user** perform **this action** on **this document**?
- Integrates with Cognito, so token claims become attributes in the decision.
- Policies live outside application code, so authorisation rules can change without a deploy.
- Distinct from IAM: IAM governs AWS API calls, Verified Permissions governs your application's own resources.

> ON THE EXAM
>
> The dividing line: IAM decides whether you may call s3:GetObject. Verified Permissions decides whether Alice may edit document 42 in your app.

Which identity service?

THE SCENARIO SAYS	THE INTENDED ANSWER
Application end users signing up and signing in	Cognito user pool
A client calling S3 or DynamoDB directly	Cognito identity pool
Employees signing in to the AWS console across accounts	IAM Identity Center
An application or AWS service needing AWS permissions	An IAM role
Per-resource rules inside your own application	Amazon Verified Permissions
Guest access before signing in	Identity pool unauthenticated role

ON THE EXAM

Six rows, and most identity questions on this exam reduce to one of them. The first two are the pair candidates confuse most.

Quotas and practical limits

- Request rates are quota-limited per API category, and heavy sign-in traffic can be throttled.
- Cognito is **Regional**. A user pool does not replicate across Regions.
- There is no built-in export of users **with passwords**, which makes leaving Cognito awkward.
- Caching tokens for their full lifetime is the single biggest cost and throttling saving.
- **Cognito Sync is legacy.** AWS AppSync is the successor for syncing application data.

> ON THE EXAM
>
> The Regional point matters for multi-Region designs: a user pool in one Region is a dependency, and there is no managed replication.

Ask first: does this application call your API, or does it call an AWS service directly?

A photo-sharing mobile app lets users sign up with email or with Google. Once signed in, the app uploads photos directly from the device to Amazon S3, and each user must be able to access only their own prefix.

The team does not want to build a user directory, and does not want uploads passing through their own servers.

What should the team use?

A. A Cognito user pool only, with the ID token sent to S3

B. A Cognito identity pool only, with users authenticating against Google

C. A Cognito user pool for sign-in, feeding an identity pool that returns credentials scoped by policy variables

D. An IAM user per app user, with keys delivered after sign-in

◆ PAUSE AND DECIDE

What should the team use?

A User pool only

S3 does not accept Cognito JWTs. It requires SigV4-signed requests with AWS credentials, which a user pool does not issue.

B Identity pool only

It would work for Google sign-in alone, and the app also needs email sign-up, which requires a directory. An identity pool holds no users.

C User pool plus identity pool

The user pool provides the directory and Google federation; the identity pool converts the token into credentials, scoped per user by policy variables.

D IAM user per app user

Module 16 ruled this out. IAM users are not for application end users, and the 5,000 user quota makes it unworkable anyway.

THE SIGNAL

Two requirements force both halves: email sign-up needs a directory, and direct S3 access needs AWS credentials. Neither service does both.

A company is moving 400,000 users from a self-hosted authentication system to a Cognito user pool. The existing database stores only bcrypt password hashes, which Cognito cannot import.

Forcing every user to reset their password has been rejected by the business. Users should notice nothing.

How should the migration be done?

A Bulk import users with the CSV import job and send everyone a password reset email

B Configure a user migration Lambda trigger that authenticates against the legacy system on first sign-in

C Store the bcrypt hashes in a custom attribute and validate them in a pre authentication trigger

D Run both systems in parallel indefinitely and route users based on a flag

◆ PAUSE AND DECIDE

How should the migration be done?

A CSV import plus resets

Exactly what the business rejected. CSV import cannot carry passwords, so every user must reset.

B User migration trigger

Built for this. The user signs in normally, the trigger validates against the old system, and Cognito creates the user with the password they typed.

C Hashes in a custom attribute

Pre authentication runs after Cognito has already tried to verify the password, so it cannot substitute its own check. It also stores hashes in a readable attribute.

D Parallel systems

Avoids the migration rather than performing it, and leaves two authentication systems to maintain forever.

THE SIGNAL

'Only hashes' plus 'no password resets' is the exact shape the migration trigger exists for. It appears whenever a scenario describes moving to Cognito.

A multi-tenant SaaS application uses a Cognito user pool behind an API Gateway REST API with a Cognito authoriser. Each Lambda function currently queries DynamoDB on every request to discover which tenant the caller belongs to, adding latency and cost.

Tenant membership changes rarely. The team wants the tenant available to the backend without a lookup, and wants it to be tamper-proof.

What should the team implement?

A Have the client send the tenant ID in a custom HTTP header

B Cache the tenant lookup in ElastiCache, keyed by user ID

C Replace the Cognito authoriser with a Lambda authoriser that does the lookup once and caches it

D Add a pre token generation trigger that injects the tenant ID as a claim

◆ PAUSE AND DECIDE

What should the team implement?

A Client-supplied header

A client-controlled value is not tamper-proof. Any caller could claim another tenant, which is a cross-tenant data breach.

B ElastiCache lookup

Faster than DynamoDB, and still a lookup on every request, plus a cache to run and invalidate.

C Lambda authoriser with caching

Genuinely works, and adds a function invocation and its own cache TTL to manage. More moving parts than putting the value in the token.

D Pre token generation trigger

The tenant becomes a signed claim. No lookup at all, and the signature makes it tamper-proof. Membership changes rarely, so token lifetime is not an issue.

THE SIGNAL

'Tamper-proof' is the deciding word. A signed token claim cannot be altered by the client; a header can. And 'changes rarely' makes token staleness acceptable.

> A document management application uses a Cognito user pool and an API Gateway Cognito authoriser. Any signed-in user can currently call any endpoint. The business needs rules such as: a user may edit a document if they own it, or if they are in the reviewers list for that document's project.
>
> These rules change frequently, and the security team wants them auditable outside application code.

How should authorisation be implemented?

A. Amazon Verified Permissions, with Cedar policies evaluated per request using token claims

B. Cognito groups, with one group per document

C. An IAM policy per user, using policy variables to match document ownership

D. Hard-code the rules in each Lambda function and deploy when they change

◆ PAUSE AND DECIDE

How should authorisation be implemented?

A Verified Permissions with Cedar

Purpose-built for per-resource application authorisation. Policies live outside code, are auditable, and change without a deployment.

B A group per document

Groups are coarse-grained and capped in number. One group per document does not scale past a few hundred documents.

C An IAM policy per user

IAM governs AWS API calls, not your application's documents. It has no concept of a reviewers list on a project.

D Rules in code

Exactly what the security team ruled out. Frequently changing rules would need a deployment each time, and nothing is auditable.

THE SIGNAL

The dividing line again: IAM decides whether you may call s3:GetObject. Verified Permissions decides whether this user may edit that document.

What to carry forward

- **User pool authenticates. Identity pool issues AWS credentials.** Direct AWS calls need both.
- **Access token for APIs, ID token for your app.** The identity pool exchange uses the ID token.
- **Lambda triggers** are the most examinable part: migrate user, pre token generation above all.
- Tokens: 1 hour by default, refresh 30 days. Enhanced identity pool flow always gives 1 hour.
- The Cognito authoriser answers 'valid user'. Per-resource rules need Verified Permissions.

> ON THE EXAM
>
> Next: AWS KMS and encryption in practice, where module 15's key policy exception finally gets the attention it deserves.